Auditory AI

Keyword Detection System

Project Team:

Mathias Kornschober (PyTorch)
Dominik Praja (TensorFlow)

Date:

June 17, 2026

Institution:

HTL Bulme
Zweig Elektronik und technische Informatik
Hochstamm Künstliche Intelligenz

Class: 4CHEL
Supervisor: Prof. DI Schamberger Rudolf

Contents

1 Introduction and GitHub Repository

Voice-controlled interfaces have traditionally relied on cloud-based speech-to-text APIs, which present several challenges, including network latency, privacy concerns, and dependencies on active internet connections. The Auditory AI project was designed to address these concerns by implementing a high-performance, completely offline, on-device keyword detection system. The primary goal of the system is to capture spoken commands under a restricted vocabulary–specifically the directional and confirmation keywords *yes*, *no*, *up*, and *down*–and map them to real-time user interface actions, such as navigation or retro arcade game control (Voice Arcade).

To achieve high accessibility, the system is designed to run entirely client-side on consumer-grade hardware. Rather than relying on a heavy desktop Python environment for inference, the application utilizes WebAssembly via `onnxruntime-web` to run pre-trained convolutional neural network (CNN) graphs directly in the browser's context. Preprocessing and feature extraction–including Short-Time Fourier Transform (STFT), Mel-scale warping, and Discrete Cosine Transform (DCT-II)–are executed in a custom, pure JavaScript digital signal processing library (`dsp.js`) engineered to align exactly with standard Python audio processing packages. This architecture enables unified deployment across multiple platforms, facilitating native installations for Windows desktop, Android, and iOS using a single, decoupled HTML5/JavaScript codebase.

This document details the complete design, comparison, and implementation logs of two distinct machine learning pipelines: a PyTorch-based CNN pipeline and a TensorFlow/Keras-based CNN pipeline. Both models are optimized, exported to the open-standard ONNX format, and packaged into the frontend. Additionally, this document log details the dataset structure, model optimization benchmarks, training challenges (such as Class Imbalance and Batch Normalization instabilities), and cross-platform native compilation details.

The complete project repository containing training notebooks, model weights, native configurations, and application source code is available on GitHub at the following link:

`https://github.com/KOFiblto/KeywordDetection`

2 Dataset Specification and Preparation

2.1 Dataset Source and Licensing

The raw audio dataset is sourced from the Kaggle mirror of the Google Speech Commands dataset (`https://www.kaggle.com/datasets/neehakurelli/google-speech-commands`). Although this specific mirror lists the license as unknown or unspecified on Kaggle, the original dataset was created and officially released by Pete Warden (Google Brain team) under the Creative Commons Attribution 4.0 International (CC BY 4.0) license. The CC BY 4.0 license explicitly permits redistribution, modification, and academic/commercial use under the condition that proper attribution is provided. Therefore, the dataset is safe and legally compliant for academic and educational project use.

2.2 Restructuring and Data Pipeline

Both PyTorch and TensorFlow pipelines train on the same shared dataset structure. An automatic download and restructuring script moves target keywords to separate directories, downsamples the negative classes, slices background noises, and creates a unified structure under a dedicated folder.

Since the raw dataset contained over 56,000 files in the `other` category (unused background noise and words), while target keywords (*yes*, *no*, *up*, *down*) only had approximately 2,000 files each, the model risked heavily over-predicting the negative class. This was resolved by downsampling the `other` category randomly to 10,000 files, splitting it in a stratified manner, and oversampling target keywords in the training split to exactly 8,000 files each using random choices with replacement. Figure 1 shows the final sample distribution.

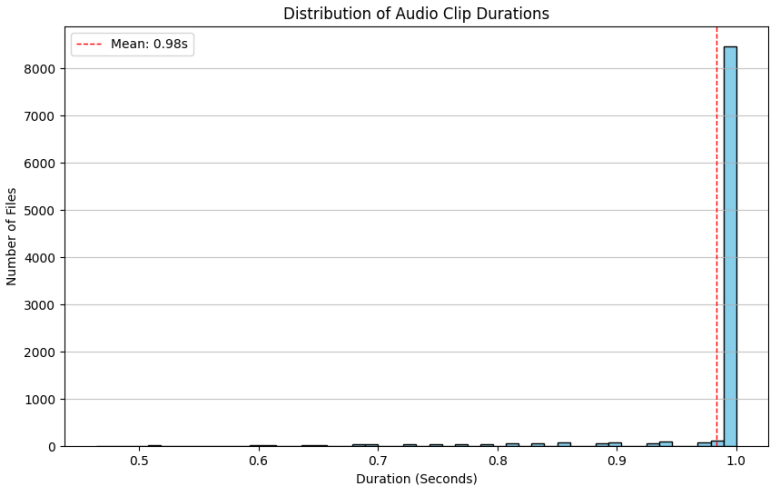

Figure 1: Distribution of audio samples across classes in the dataset

To filter out silent segments, ambient noise, and mislabeled utterances from the raw dataset, the project utilizes a self-written manual review application located in the `Utils/WavCleanUp` directory. This custom graphical tool provides an interface to review WAV file waveforms, calculate signal energy, and play segments, allowing developers to manually audit and prune low-quality audio files from target directories before the dataset is processed by the training pipeline.

The restructured dataset is partitioned into a stratified 3-way split consisting of 70% training, 10% validation, and 20% test subsets to evaluate generalization performance.

3 PyTorch and TensorFlow Model Pipelines

3.1 Preprocessing and Feature Transformation

Rather than feeding raw audio waveforms directly into the convolutional layers during inference, both pipelines utilize a structured feature representation. Preprocessing steps are executed in a sliding window layout:

1. **Resampling**: Waveforms are resampled from the capture rate (e.g. 44.1 kHz or 48 kHz) to a standard 16,000 Hz mono channel.
2. **Short-Time Fourier Transform (STFT)**: Audio signals are divided into overlapping spectral frames.
3. **Mel-Scale Warping**: Spectrogram frequencies are mapped onto Mel filterbanks (64 bins) to mimic human auditory perception.
4. **Logarithmic Scaling**: Computes the logarithm of the Mel energies to align with human loudness perception.
5. **Discrete Cosine Transform (DCT-II)**: The first 40 coefficients are extracted to obtain Mel-Frequency Cepstral Coefficients (MFCCs), decorrelating adjacent frequency bands.

Figure 2 illustrates the visual representation of these features side-by-side.

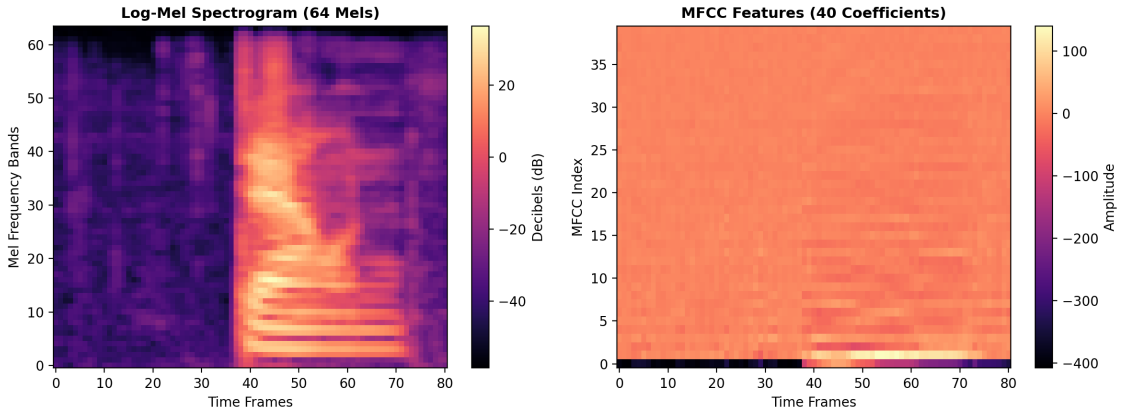

Figure 2: Log-Mel Spectrogram and MFCC features extracted from a sample audio clip

3.2 Model Architectures

The architectures of the PyTorch and TensorFlow networks are shown in Tables 1 and 2.

3.3 ONNX Export

Compiling and exporting both models to the ONNX format was done to enable framework-independent deployment. By packaging the weights and network graph into a standardized ONNX representation, the production frontend can execute deep learning inference directly via WebAssembly (`onnxruntime-web`). This eliminates heavy Python library dependencies (such as PyTorch or TensorFlow) in production, yielding a client-side execution latency under 5 ms.

3.4 Model Optimization and Milestones (PyTorch)

The PyTorch model was optimized progressively across 11 training iterations, as documented in Table 3.

Table 1: PyTorch CNN Model Structural Layout

Layer Name	**Type**	**Filters / Units**	**Activation / Parameters**
Input	-	-	Input size: $(1, 40, 81)$
Conv 1	2D Convolution	16	ReLU, 3×3, padding 1
Pool 1	Max Pooling	-	2×2 pool
Conv 2	2D Convolution	32	ReLU, 3×3, padding 1
Pool 2	Max Pooling	-	2×2 pool
Conv 3	2D Convolution	64	ReLU, 3×3, padding 1
Pool 3	Max Pooling	-	2×2 pool
Conv 4	2D Convolution	128	ReLU, 3×3, padding 1
Pool 4	Max Pooling	-	2×2 pool
Adaptive Pool	Adaptive Avg Pool	-	Downscale to 4×4
Flatten	Flatten	-	Flatten to 2048
FC 1	Fully Connected	128	ReLU, Dropout ($p = 0.3$)
FC 2 (Output)	Fully Connected	5	Linear output

Table 2: TensorFlow / Keras CNN Model Structural Layout

Layer Name	**Type**	**Filters / Units**	**Activation / Parameters**
Input	-	-	Input size: $(40, 81, 1)$
Conv 1	2D Convolution	16	ReLU, BN, 3×3, padding same
Pool 1	Max Pooling	-	2×2 pool
Conv 2	2D Convolution	32	ReLU, BN, 3×3, padding same
Pool 2	Max Pooling	-	2×2 pool
Conv 3	2D Convolution	64	ReLU, BN, 3×3, padding same
Pool 3	Max Pooling	-	2×2 pool
Conv 4	2D Convolution	128	ReLU, BN, 3×3, padding same
Pool 4	Max Pooling	-	2×2 pool
Resizing	Resizing	-	Resize to 4×4
Flatten	Flatten	-	Flatten to 2048
Dense 1	Dense	128	ReLU
Dropout	Dropout	-	Dropout ($p = 0.3$)
Dense 2 (Output)	Dense	5	Linear output

Table 3: Chronological training and evaluation path

Iter.	Script Name	Accuracy	Result and Observation
01	`01_specaugment.py`	95.10 %	Solid baseline using frequency and time masking.
02	`02_audio_data_augmentation.py`	95.52 %	Random time-shifting improves stability.
03	`03_lr_scheduler.py`	95.68 %	ReduceLROnPlateau scheduler refines convergence.
04	`04_batch_normalization.py`	53.95 %	Severe drop (SpecAugment conflict with BatchNorm).
05	`05_weight_decay.py`	94.89 %	L2 weight decay regularizer applied.
06	`06_increase_filters_and_mfcc.py`	96.79 %	Significant boost by switching to 40 MFCCs.
07	`07_increase_mel_bins.py`	89.67 %	Overfitting after increasing Mel bins to 128.
08	`08_increase_dropout.py`	95.52 %	Dropout (0.3) helps regularize filters.
09	`09_model_checkpointing.py`	95.15 %	Checkpoint loading saves the best epoch parameters.
10	`10_combined_best.py`	97.95 %	Combined best configurations from 1–9.
11	`11_combined_stable.py`	**98.52 %**	Standardized parameters for stable production.

Iteration 10 (`10_combined_best.py`) was designed to combine the most successful configurations identified during Iterations 01 to 09. This included utilizing 40 MFCC coefficients, ReduceLROnPlateau scheduler parameters, time-shifting audio augmentations, and optimal SpecAugment boundaries. Iteration 11 (`11_combined_stable.py`) further standardized these features to yield the best stable accuracy for production deployment.

3.5 Training and Evaluation Metrics

The final training and testing metrics of the two pipelines are compared in Table 4.

Table 4: Comparison of training and testing metrics for PyTorch and TensorFlow

Metric	PyTorch (CNN)	TensorFlow (Keras CNN)
Trained Epochs	40 / 40	100 / 100
Final Training Loss	0.0858	0.0515
Final Training Accuracy	96.86 %	98.32 %
Final Validation Accuracy	—	97.25 %
Best Test/Validation Accuracy	98.52 %	97.46 %
Final Test Accuracy	98.47 %	97.46 %
Average Epoch Time (GPU)	approx. 4.50 s	approx. 3.20 s

The training and validation accuracy progression of the PyTorch model over the training epochs is illustrated in Figure 3.

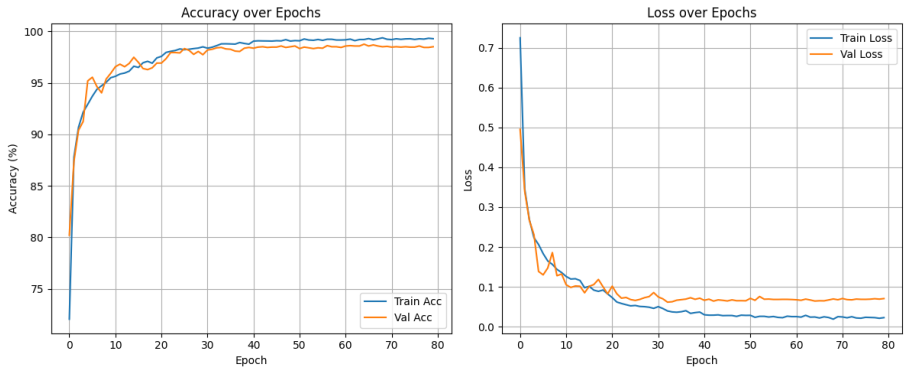

Figure 3: PyTorch training and validation accuracy progression over epochs

3.6 Classification Results (Confusion Matrices)

Figures 4 and 5 represent the confusion matrices of both pipelines evaluated against the validation test data.

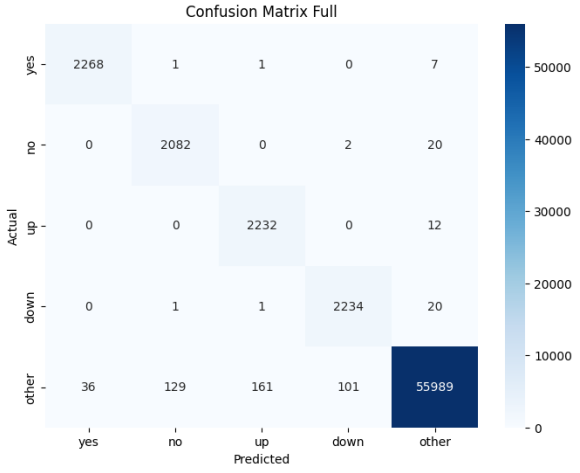

Figure 4: PyTorch final model confusion matrix

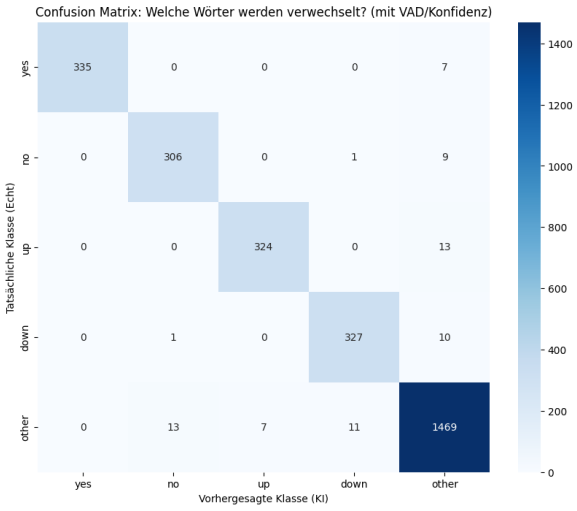

Figure 5: TensorFlow final model confusion matrix

The matrices indicate that both models display high validation separation between target command keywords, with minimal cross-classification error.

3.7 Challenges and Resolutions

3.7.1 1. Batch Normalization Crash in PyTorch

SpecAugment zero-masks entire frequency/time bands. In the PyTorch pipeline, applying Batch Normalization concurrently caused unstable running standard deviation and mean metrics, crashing validation accuracy to 53.95%. Discarding BatchNorm for the shallow CNN solved the crash. In Keras, BatchNorm was successfully integrated because SpecAugment is only evaluated inside the training dataset pipeline and bypassed during validation model calls.

3.7.2 2. Dynamic ONNX Shape Transposition

PyTorch models expect a channel-first dimension layout of `[Batch, Channel, Height, Width]` (e.g., `[1, 1, 40, 81]`), whereas TensorFlow/Keras models expect a channel-last layout of

`[Batch, Height, Width, Channel]` (e.g., `[1, 40, 81, 1]`). To maintain a unified client-side inference loop in JavaScript, the application dynamically inspects the loaded ONNX model's input signature shape. If the final dimension of the 4D shape is detected as 1, the system automatically builds a channel-last tensor layout; otherwise, it defaults to a channel-first layout. This runtime validation ensures cross-model compatibility without hardcoding model-specific inference logic.

3.7.3 3. Poor Data Meticulosity in the Raw Dataset and Interactive Cleanup

The Google Speech Commands raw dataset contained multiple quality defects, such as empty, silent, corrupt, or mislabeled audio files. Empty files crashed loader pipelines, while silent files caused constant false triggers in production. A custom PyQt6-based cleanup tool (`Wav_File_Cleanup.py`) was developed to calculate the RMS signal energy of all files and sort them. Quiet or corrupt files are highlighted to the developer and can be deleted or reclassified in split seconds using keyboard shortcuts.

3.7.4 4. Extreme Class Imbalance

Since the `other` category contained over 56,000 files (unused background noise and words), while target keywords (*yes*, *no*, *up*, *down*) only had 2,000 files each, the model risked heavily over-predicting the negative class. This was solved by downsampling the `other` category randomly to 10,000 files, splitting it in a stratified manner (70 % train, 15 % val, 15 % test), and oversampling target keywords in the training split to exactly 8,000 files each using random choices with replacement.

3.7.5 5. CPU Bottleneck during Disk I/O in Training (Resolving Memory and Disk I/O Bottlenecks)

Initial training runs suffered from severe disk read bottlenecks: CPU usage spiked to 70–100 % while loading files dynamically, keeping the GPU idle and starved of data 70 % of the time. This was resolved by implementing a RAM Caching strategy inside the dataset loader. All audio files are loaded once during initialization, mono-averaged, padded/cropped, and stored directly in RAM. Dynamic time-shifting is computed on-the-fly on the cached RAM tensors. Epoch training duration dropped from minutes to under 5 seconds.

3.7.6 6. Voice Activity Detection (VAD) and Confidence Filtering

In continuous sliding window mode (every 200 ms), environmental typing, clicking, and breathing caused false triggers. To combat this, two filters are applied client-side:

- **Voice Activity Detection (VAD)**: If the calculated root-mean-square (RMS) amplitude of the 1-second buffer falls below 0.002, the prediction defaults to `other` without evaluating the ONNX graph.
- **Confidence Filtering**: If the argmax softmax probability of the model output is lower than 85%, the detection is discarded.

3.7.7 7. Hardware Resource Locking in the Web Browser (Audio Stream Locking)

When hot-plugging microphones or changing input devices in the Electron GUI, the old device stream was frequently locked by the browser, causing hardware conflicts in Windows. This was resolved by iterating over all active `MediaStreamTrack` objects of the current audio stream and explicitly calling `track.stop()` to release system resources before initializing the new device stream.

4 Client-Side Web Application

4.1 Client-Side Pure JS Architecture

To eliminate backend latency and deployment constraints, the application is fully client-side. Preprocessing and machine learning inference are performed directly inside the browser's context or web view:

- **Audio Capture & Buffer**: Captures raw audio via the browser's audio API. In Live Mode, raw samples are continuously pushed into a circular float array matching the native sample rate (e.g., 44.1 kHz or 48 kHz).
- **Resampling**: Every 200 ms, the last 1.0 second of raw audio is read and downsampled to 16,000 Hz using a linear interpolation helper in JavaScript.
- **JS Feature Extraction**: The resampled samples are transformed into MFCC (v1 models) or Log-Mel Spectrogram (v2 models) feature maps using JavaScript digital signal processing functions implemented in `dsp.js`.
- **WASM Inference**: The feature maps are evaluated by loading the model from the local assets folder into a web-based runtime utilizing WebAssembly.

4.2 Interface & Visuals

The interface is styled using responsive CSS dark theme variables. Real-time visualizations are rendered at 60 FPS on HTML5 canvases:

1. **Oscilloscope**: Renders time-domain audio amplitude.
2. **Scrolling Spectrogram**: Renders frequency components scrolling over a 15-second history.
3. **Timeline**: Overlays color-coded detection blocks (YES: Mint Green, NO: Soft Coral, UP: Soft Blue, DOWN: Slate Blue, OTHER: Pastel Yellow) synced with the audio.

4.3 Voice Arcade

Contains five lightweight canvas games controlled by voice:

1. **Flappy Bird**: Directional jumps.
2. **Grid Runner**: Collect gems using directions.
3. **Space Defender**: Shooter mapped to directions and shields.
4. **Simon Memory**: Follow and repeat directional sequences.
5. **Cyber Hi-Lo**: Card guessing game.

Game physics are modified dynamically via a speed multiplier slider to offset vocal processing delays. The games utilize a procedural synthesizer (oscillators and gain nodes in Web Audio API) to synthesize retro sound effects locally.

5 Installation and Local Setup Guide

5.1 System Setup Requirements

To configure and run the Auditory AI local development environment, ensure the following tools are installed on the host system:

- **Node.js & npm** (version 18 or newer)
- **Git**
- **Python** (version 3.10 to 3.12). Python 3.13 is incompatible with TensorFlow/Keras and should be avoided. A single local Python installation can support both pipelines.

First, clone the repository and change directory to the project root:

```
git clone https://github.com/K0Fiblto/KeywordDetection.git
cd KeywordDetection
```

5.2 Configure PyTorch Pipeline Environment

To run training files, audit directories, and evaluate PyTorch classifiers, establish a dedicated virtual environment and install the package requirements:

```
python -m venv .venv
# Under PowerShell:
.\.venv\Scripts\Activate.ps1
# Under CMD:
.\.venv\Scripts\activate.bat

pip install -r install/pytorch/pytorch-requirements.txt
```

The dataset restructuring script can be executed next to download and prepare the Google Speech Commands files:

```
python install/Download_Dataset.py
```

5.3 Configure TensorFlow Pipeline Environment

To compile and test Keras models, create a separate virtual environment to isolate library files, then install the TensorFlow packages and the ONNX conversion utilities:

```
python -m venv .venv_tf
# Under PowerShell:
.\.venv_tf\Scripts\Activate.ps1
# Under CMD:
.\.venv_tf\Scripts\activate.bat

pip install -r install/tensorflow/tensorflow-requirements.txt
```

5.4 Configure Frontend Node.js Dependencies

Install the client-side packages required to launch, run, and wrap the application:

```
cd frontend
npm install
cd ..
```

5.5 Running the Application Locally

To execute the application, run the automated startup batch file from the root folder:

```
start.bat
```

This batch script calls the Node.js startup service runner (`start-services.js`), which launches the Vite server, mounts assets, and spawns the Electron app window.

If required, port conflicts and background services can be terminated by executing the cleanup batch file:

```
stop.bat
```

6 Native Releases and Release Pipeline

Auditory AI supports native builds and cross-platform compilation of standalone installation packages:

- **Windows release package**: Executable desktop applications can be built via:

```
cd frontend
npm run dist-app
```

  This uses the `electron-builder` framework to bundle the application assets, local configurations, and ONNX models into a standalone installer (.exe) output to the `dist-app/` folder.

- **Mobile native projects**: Compilation is synced to native mobile directories using Capacitor. To sync the web assets:

```
npm run build
npx cap sync android
npx cap sync ios
```

  For Android, running `npx cap open android` opens the native package inside Android Studio. For iOS, running `npx cap open ios` opens the workspace in Xcode (macOS only).

- **CI/CD Release Automation**: In the production release pipeline, the Android APK compilation is executed automatically in a GitHub Actions runner using Gradle and the Android SDK command-line tools. As a result, local installations of Android Studio or Xcode are entirely optional for deployment and are only needed if developers intend to run local emulator testing.

Pre-compiled installation files, target native builds, and release assets are published and accessible on the project's GitHub Releases page:

`https://github.com/KOFiblto/KeywordDetection/releases`

7 Practical Insights and Lessons Learned

7.1 Advantage of MFCCs over Mel-Spectrograms

Standard Mel-spectrograms retain high correlation between adjacent frequency bands. By applying the Discrete Cosine Transform (DCT) to obtain MFCCs, these bands are decorrelated. This yields a highly compact representation of speech formants, enabling the network to differentiate vocal characteristics from background noise.

7.2 Handling Noise and SpecAugment

Frequency masking is crucial for robust speech recognition. It forces the network to learn global word patterns even when specific bands are attenuated by cheap microphones or ambient noise. In combination with random time-shifting in the data loader, this achieves superior generalization during real-time live capture.

7.3 Platform-Independent Inference via ONNX

Compiling models to ONNX eliminated large deep learning library dependencies in production. Inference latency dropped to under 5 ms per pass on a CPU, enabling highly resource-efficient deployments.